Full Explanation: Customizing Spring Security Configuration

When you initially add the Spring Security dependency to a Spring Boot project, it automatically secures all endpoints and provides a default login form. However, in a real-world application, you rarely want to rely on these defaults. You will need to customize the security filter chain to connect to a database, secure specific REST APIs, handle sessions differently, or disable features like CSRF (Cross-Site Request Forgery) protection for stateless applications.

To achieve this, Spring Security requires you to create a dedicated configuration class where you define a custom SecurityFilterChain bean. By doing this, you tell Spring to bypass its default security rules and follow your explicit instructions.

Detailed Notes & Core Concepts

- **The Configuration Class:**
- To customize Spring Security, you create a standard Java class (e.g., SecurityConfig) and annotate it with @Configuration.
- You must also use @EnableWebSecurity to tell Spring Boot to disable the default web security configuration and apply your custom rules instead.
- **The Security Filter Chain:**
- Spring Security works by passing incoming requests through a chain of filters. To alter this chain, you define a method that returns a SecurityFilterChain object and annotate it with @Bean.
- **The HttpSecurity Object:**
- This is the core object used to configure web-based security for specific HTTP requests. You use it to define what is protected, what is public, and how authentication should be handled.
- **Key Customizations Explained:**

1. **Disabling CSRF:** http.csrf(customizer -> customizer.disable()). CSRF protection is useful for traditional web apps but is often disabled for stateless REST APIs.
2. **Authorizing Requests:** http.authorizeHttpRequests(request -> request.anyRequest().authenticated()). This ensures that every single request coming into the application requires authentication.
3. **Form Login vs. HTTP Basic:**
   - formLogin() provides a standard HTML login page for browser users.
   - httpBasic() allows tools like Postman (or front-end frameworks like React/Angular) to authenticate by passing credentials in the HTTP header.

1. **Session Management:** By setting the session creation policy to STATELESS, Spring Security stops generating session IDs for users. This is standard practice for modern REST APIs (often paired with JWTs) where the client must prove its identity on every single request.

- **The Builder Pattern:**
- Instead of calling methods on the HttpSecurity object on separate lines, you can chain them together (e.g., http.csrf(...).authorizeHttpRequests(...)). This makes the code cleaner and easier to read.

Example Code Section

Below is the complete, modernized implementation of the security configuration using the Builder pattern and Lambda expressions, as discussed in the transcript.

Java

```
package com.example.config;import org.springframework.context.annotation.Bean;import
org.springframework.context.annotation.Configuration;import
org.springframework.security.config.Customizer;import
org.springframework.security.config.annotation.web.builders.HttpSecurity;import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;import
org.springframework.security.config.http.SessionCreationPolicy;import
org.springframework.security.web.SecurityFilterChain;@Configuration@EnableWebSecuritypublic
class SecurityConfig {    @Bean    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {                return http            // Disable CSRF for stateless REST
APIs            .csrf(customizer -> customizer.disable())                    // Require authentication for all
incoming requests            .authorizeHttpRequests(request ->
request.anyRequest().authenticated())                    // Enable HTTP Basic Authentication for tools
like Postman            .httpBasic(Customizer.withDefaults())                    // Make the application
stateless (no session ID will be stored or tracked)            .sessionManagement(session
->                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
  // Build and return the configured filter chain            .build();    }}
```

Expected Outputs & Error Handling

Here is what happens when the configuration is correct, versus when common mistakes are made.

1. Expected Output (Correct Code)

- **Browser Access:** If you visit the application in a browser, instead of an HTML form, you will get a browser-native popup asking for a username and password (because formLogin is removed and httpBasic is enabled).
- **Postman GET/POST Requests:** If you send a request without credentials, you will receive a 401 Unauthorized status. If you go to the "Authorization" tab in Postman, select "Basic Auth", and provide the correct credentials, you will receive a 200 OK and the actual JSON/data response.

2. What happens if the code is wrong?

- **Mistake:** Forgetting to add @Configuration or @EnableWebSecurity to the class.
- **Output:** Spring completely ignores your custom file. You will fall back to the default Spring Security behavior (a default HTML login form will appear for all endpoints).
- **Mistake:** You want to test APIs in Postman, but you only configure http.formLogin() and forget http.httpBasic().
- **Output:** You will send a GET request from Postman and get a 200 OK status, but the response body will contain raw HTML code. This happens because Spring is successfully redirecting Postman to the HTML login page instead of returning your API data.
- **Mistake:** Forgetting to define http.authorizeHttpRequests(...).
- **Output:** The application becomes entirely public. Any user can access any endpoint without providing a username or password.
- **Mistake:** Sending a POST request from Postman without disabling CSRF (http.csrf().disable()).
- **Output:** You will receive a 403 Forbidden error. Even if you provide the correct username and password, Spring will block the POST request because it is expecting a valid CSRF token, which Postman does not automatically have.

Spring Framework එකේ ගොඩක් වැදගත් **annotations** දෙකක් තමයි `@Configuration` සහ `@Bean` කියන්නේ. අපි කලින් කතා කරපු Spring Security උදාහරණයේදීත් ඔයාට මේවා දකින්න ලැබෙන්න ඇති. මේවායින් කරන්නේ අපේ application එකට අවශ්‍ය objects (Spring වලදී අපි මේවාට කියන්නේ **Beans** කියලා) හදන එක සහ ඒවා manage කරන එක Spring වලට බාර දෙන එකයි.

සරල උදාහරණයක් විදිහට, Spring application එකක් කියන්නේ ලොකු කර්මාන්තශාලාවක් (Factory) කියලා හිතන්න.

1. `@Configuration` (උපදෙස් පත්‍රිකාව / Blueprint)

මේ annotation එක අපි පාවිච්චි කරන්නේ class එකක් උඩින්. මේකෙන් Spring වලට කියන්නේ, *"මේ class එක ඇතුලේ තියෙන්නේ application එකට අවශ්‍ය වෙන objects හදන විදිහ ගැන උපදෙස්"* කියලයි.

- `@Configuration` දාලා තියෙන class එකක් දැක්කම, Spring ඒ class එක ඇතුලට ගිහින් බලනවා මොනවද හදන්න ඕනේ **Beans** කියලා.

2. `@Bean` (නිෂ්පාදනය / The Product)

මේ annotation එක අපි පාවිච්චි කරන්නේ `@Configuration` class එකක් ඇතුලේ තියෙන method එකක් උඩින්. මේකෙන් Spring වලට කියන්නේ, *"මේ method එක run කරලා ඒකෙන් return කරන object එක අරගෙන, ඔයාගේ **IoC Container** එක ඇතුලේ තියාගෙන manage කරන්න"* කියලයි.

- එතකොට application එකේ වෙන කොහේ හරි තැනක ඒ object එක පාවිච්චි කරන්න ඕනේ වුණාම (උදාහරණයක් විදිහට **Dependency Injection** හරහා), Spring විසින් අර කලින් හදලා තියාගත්ත object එක අරගෙන දෙනවා. අලුතින් objects හැමතිස්සෙම හදන්න ඕනේ වෙන්නේ නැහැ.

සරල Code උදාහරණයක්

```Java
import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;


// Marks this class as a source of bean definitions

@Configuration

public class DatabaseConfig {


    // Tells Spring to call this method and manage the returned object as a Bean

    @Bean

    public DatabaseConnection myDatabaseConnection() {
```

```
        DatabaseConnection connection = new DatabaseConnection();
        connection.setUrl("jdbc:mysql://localhost:3306/mydb");
        connection.setUsername("root");
        connection.setPassword("password123");

        // Spring will take this returned object and keep it in the application context
        return connection;
    }
}
```

මේකෙන් වෙන දේ කෙටියෙන්:

ඔයා සාමාන්‍යයෙන් Java වලදී අලුත් object එකක් හදන්නේ `new DatabaseConnection()` කියලා නේ. හැබැයි Spring වලදී අපි කැමතියි ඒ objects හදන එකයි, ඒවා එකිනෙකට සම්බන්ධ කරන එකයි Spring වලටම කරන්න දෙන්න. `@Configuration` සහ `@Bean` පාවිච්චි කරලා අපි Spring එකට ඒ දේවල් කරන්න අවශ්‍ය මඟ පෙන්වීම ලබා දෙනවා.

හොඳ ප්‍රශ්නයක්! අපි දැන් බලමු ඔයා අහපු ඒ `@Configuration` සහ `@Bean` කියන සංකල්ප දෙක, අර **CSRF** disable කරපු code section එකේදී ප්‍රායෝගිකව වැඩ කරන්නේ කොහොමද කියලා.

අපි අර කලින් කතා කරපු code එකේ කොටසක් විතරක් අරගෙන බලමු:

```Java
@Configuration // 1. උපදෙස් පත්‍රිකාව
@EnableWebSecurity
public class SecurityConfig {

    @Bean // 2. නිෂ්පාදනය
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
```

```
        return http
            // 3. Object එක අපිට ඕන විදිහට හැඩගැන්වීම
            .csrf(customizer -> customizer.disable())
            .build(); // 4. අවසන් Object එක සාදා නිම කිරීම
    }
}
```

මෙතනදී පියවරෙන් පියවර වෙන දේ මෙහෙමයි:

1. `@Configuration` මඟින් Spring එකට පණිවිඩයක් දෙනවා

Spring application එක start වෙද්දී එයා `@Configuration` තියෙන `SecurityConfig` class එක දකිනවා. එතකොට Spring තේරුම් ගන්නවා *"ආ.. මේ class එක ඇතුලේ මට manage කරන්න ඕනේ කරන අලුත් objects (Beans) හදන විදිහ ගැන උපදෙස් තියෙනවා"* කියලා.

2. `@Bean` මඟින් Method එක Run කරනවා

Spring ඊළඟට ඒ class එක ඇතුලට ගිහින් බලද්දී දකිනවා `securityFilterChain` කියන method එක උඩින් `@Bean` කියලා දාලා තියෙනවා. ඒක දැක්ක ගමන් Spring කරන්නේ, application එක run වෙන්න පටන් ගනිද්දීම මේ method එක ස්වයංක්‍රීයවම (automatically) call කරන එකයි.

3. Method එක ඇතුලේදී CSRF Disable වෙනවා

මේ method එක ඇතුලේදී අපි කරන්නේ Spring වලට ඕනේ කරන `SecurityFilterChain` කියන object එක අපිට ඕන විදිහට හදන එකයි.

එතනදී අපි `http.csrf(customizer -> customizer.disable())` කියලා ලියලා තියෙනවා. මේකෙන් කරන්නේ අර හැදීගෙන එන අලුත් security object එකට නීතියක් දානවා *"ඔයාගේ CSRF protection කියන feature එක වැඩ කරන්න එපා (disable කරන්න)"* කියලා.

4. අලුත් Object එක Spring වලට බාර දෙනවා

අන්තිමට `.build()` කියලා call කරාම, අපි අර දුන්න නීති ඔක්කොම (CSRF disable කිරීම ඇතුළුව) අඩංගු වුණු සම්පූර්ණ `SecurityFilterChain` object එකක් හැදිලා return වෙනවා.

`@Bean` annotation එක පාවිච්චි කරලා තියෙන නිසා, Spring කරන්නේ **අපි මේ return කරපු අලුත් object එක එයාගේ භාරයට අරගෙන**, Spring Security එකේ default තිබ්බ setting අයින් කරලා, අපි හදලා දුන්න මේ object එක මුළු application එකේම security එක control කරන්න පාවිච්චි කරන එකයි.

කෙටියෙන්ම කිව්වොත්:

`@Bean` එකක් විදිහට මේ method එක ලිව්වේ නැත්නම්, Spring කවදාවත් මේ method එක run කරන්නේ නැහැ. එතකොට අපි කොච්චර `csrf().disable()` කියලා code එක ලිව්වත් ඒක application එකට apply වෙන්නේ නැහැ. `@Bean` හරහා තමයි අපි Spring වලට බල කරන්නේ *"මේක run කරලා, මේකෙන් එන object එක පාවිච්චි කරපන්"* කියලා.

පැහැදිලියි නේද මේ process එක වෙන විදිහ? තව මොනවා හරි තේරුණේ නැති තැනක් තියෙනවද?

ඔව්, හරියටම හරි! ඔයා තේරුම් ගන්න විදිහ ගොඩක් දුරට නිවැරදියි. හැබැයි එතන පොඩි වැදගත් තිතක් තියෙනවා අපි පැහැදිලි කරගන්න ඕනේ.

ඔයා කිව්ව කතාවේ කොටස් දෙකක් තියෙනවා:

1. Spring විසින් මේවා "මුලින්ම" (First) Run කරගන්නවාද?

ඔව්, අනිවාර්යයෙන්ම. Spring Boot application එකක් start වෙද්දීම, ඒ කියන්නේ run වෙන්න පටන් ගත්ත ගමන්ම එයා කරන්නේ මුලින්ම `@Configuration` කියලා දාලා තියෙන classes හොයාගෙන යන එකයි.

ඒකට හේතුව තමයි, application එකේ අනිත් දේවල් (උදාහරණයක් විදිහට අපේ REST APIs තියෙන Controllers, දත්ත ගබඩා කරන Repositories වගේ දේවල්) වැඩ කරන්න පටන් ගන්න කලින්, මුළු application එකටම අවශ්‍ය කරන අත්තිවාරම හරි, උපකරණ (Objects/Beans) ටික හරි ලෑස්ති කරලා තියෙන්න ඕනේ. ඒ නිසා Spring මුලින්ම මේ Configuration classes ඇතුලට ගිහින් ඒකෙ තියෙන `@Bean` methods ටික run කරලා අවශ්‍ය objects ටික හදාගන්නවා.

2. මේවා "හැම වෙලාවෙම" (Every time) Run වෙනවාද?

නැහැ, හැම වෙලාවෙම run වෙන්නේ නැහැ. මෙතන තමයි ගොඩක් අයට පැටලෙන තැන.

Spring වල Objects (Beans) හදන default ක්‍රමය තමයි **"Singleton"** කියන්නේ.

ඒ කියන්නේ application එක start වෙනකොටම අර `@Bean` method එක **එක පාරක් විතරක්** run කරලා ඒ අදාළ object එක හදාගන්නවා. ඊටපස්සේ ඒ හදාගත්ත එකම object එක Spring එයාගේ memory එකේ (අපි ඒකට කියනවා **IoC Container** එක කියලා) පරිස්සමට තියාගන්නවා.

ඊටපස්සේ user කෙනෙක් web browser එකෙන් හරි Postman එකෙන් හරි request එකක් එවන **හැම වෙලාවෙම** අර method එක ආයේ ආයේ run වෙන්නේ නැහැ. Spring කරන්නේ අර මුලින්ම එක පාරක් හදලා තියාගත්ත object එකම දිගටම පාවිච්චි කරන එකයි.

කෙටියෙන් මතක තියාගන්න:

හරියට කර්මාන්තශාලාවක් (Factory) open කරන්න කලින්, ඒකට ඕනේ කරන මැෂින් ටික (Beans) ඔක්කොම එක පාරක් හයි කරලා ලෑස්ති කරගන්නවා වගේ වැඩක් තමයි මේක. කස්ටමර්ස්ලා එන එන හැම වෙලාවෙම අලුතින් මැෂින් හදන්නේ නැහැ, අර කලින් හදලා තියාගත්ත මැෂින් වලින් තමයි දිගටම වැඩ කරන්නේ. මේ Beans වැඩ කරන්නෙත් අන්න ඒ විදිහටයි.